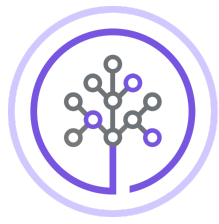


Hands-on Lab: Monitoring a DAG



Skills
Network

Estimated time needed: **20** minutes

Introduction

In this lab, you will work with the Airflow Web UI and CLI to explore the DAGs further. You will be exposed to using the interactive tools to search for DAGs, introduces to various views of the DAGS and how you can use this to explore the DAG workflow, the individual tasks in the workflow and view the outcome of the tasks.

Objectives

After completing this lab you will be able to:

- Search for a DAG
- Pause/Unpause a DAG
- Get the Details of a DAG
- Explore grid view of a DAG
- Explore graph view of a DAG
- Explore Calendar view of a DAG
- Explore Task Duration view of a DAG
- Explore Details view of a DAG
- View the source code of a DAG

- Delete a DAG

About Skills Network Cloud IDE

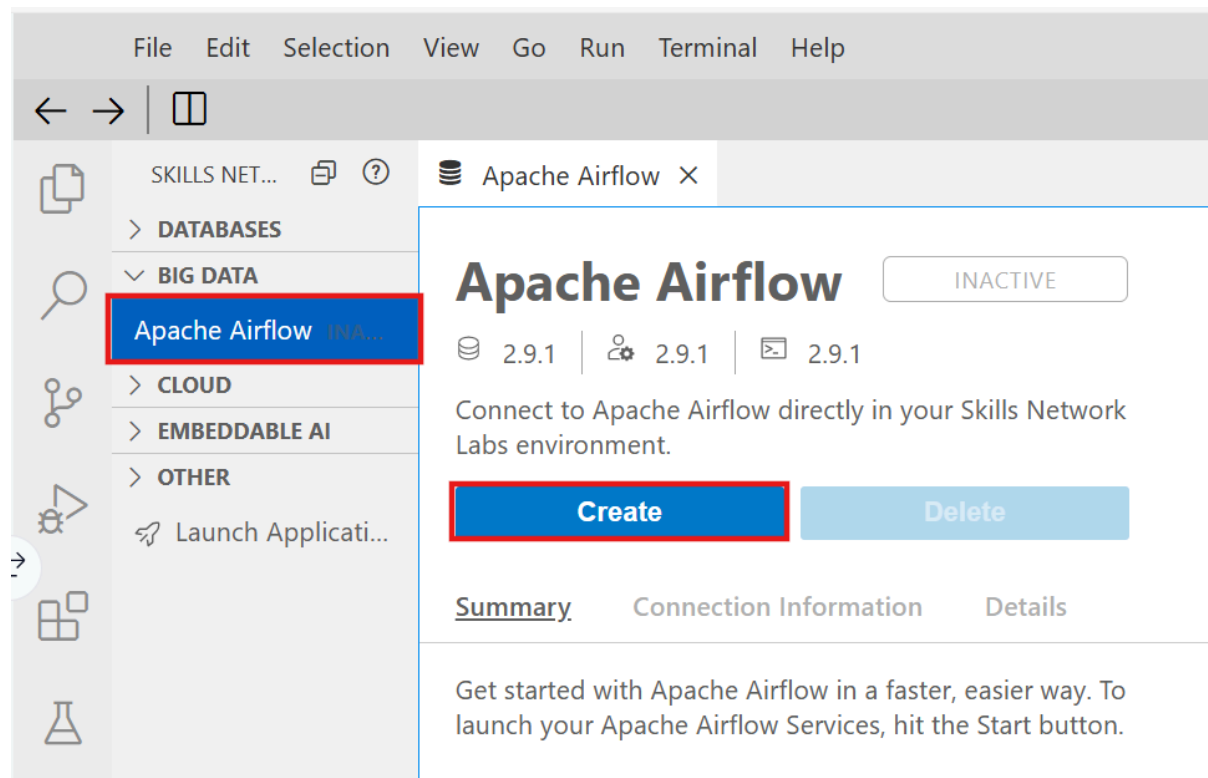
Skills Network Cloud IDE (based on Theia and Docker) provides an environment for hands on labs for course and project related labs. Theia is an open source IDE (Integrated Development Environment), that can be run on desktop or on the cloud. to complete this lab, we will be using the Cloud IDE based on Theia running in a Docker container.

Important notice about this lab environment

Please be aware that sessions for this lab environment are not persistent. A new environment is created for you every time you connect to this lab. Any data you may have saved in an earlier session will get lost. To avoid losing your data, please plan to complete these labs in a single session.

Exercise 1: Start Apache Airflow

1. Click on **Skills Network Toolbox**.
2. From the **BIG DATA** section, click **Apache Airflow**.
3. Click **Create** to start the Apache Airflow.



Note: Please be patient, it will take a few minutes for Airflow to get started.

Exercise 2: Open the Airflow Web UI

When Airflow starts successfully, you should see an output similar to the one below. Once **Apache Airflow** has started, click on the highlighted icon to open **Apache Airflow Web UI** in the new window.

Apache Airflow

ACTIVE

2.9.1 | 2.9.1 | 2.9.1

Connect to Apache Airflow directly in your Skills Network Labs environment.

StartStop

SummaryConnection InformationDetails

Your Apache Airflow Services are now ready to use and available with the following login credentials. For more details on how to navigate Apache Airflow, please check out the Details section.

Username:

Password:

You can manage Apache Airflow via:

Airflow Webserver 

You should land at a page that looks like this.

Airflow										03:53 UTC		Log In
DAGs		Cluster Activity		Datasets	Security	Browse	Admin	Docs				
☐	example_bash_operator	example	example2	airflow		00:00:00			2024-05-29, 00:00:00			

Exercise 3: Submit a dummy DAG

For the purpose of monitoring, let's create a dummy DAG with three tasks.

- Task1 does nothing but sleep for 1 second.
- Task2 sleeps for 2 seconds.
- Task3 sleeps for 3 seconds.

This DAG is scheduled to run every 1 minute.

1. Using Menu-> **File** -> **New File** create a new file named **dummy_dag.py**.
2. Copy and paste the code below into it and save the file.



python

```
# import the libraries

from datetime import timedelta
# The DAG object; we'll need this to instantiate a DAG
from airflow import DAG
# Operators; we need this to write tasks!
from airflow.operators.bash_operator import BashOperator
# This makes scheduling easy
from airflow.utils.dates import days_ago

#defining DAG arguments

# You can override them on a per-task basis during operator initialization
default_args = {
    'owner': 'Your name',
    'start_date': days_ago(0),
    'email': ['your email'],
    'retries': 1,
    'retry_delay': timedelta(minutes=5),
}

# defining the DAG
dag = DAG(
    'dummy_dag',
    default_args=default_args,
    description='My first DAG',
    schedule_interval=timedelta(minutes=1),
)

# define the tasks

# define the first task

task1 = BashOperator(
    task_id='task1',
    bash_command='sleep 1',
    dag=dag,
)

# define the second task
task2 = BashOperator(
    task_id='task2',
    bash_command='sleep 2',
    dag=dag,
)

# define the third task
task3 = BashOperator(
```

```
dag=dag,  
)  
  
# task pipeline  
task1 && task2 && task3
```

3. Set the `AIRFLOW_HOME` directory.

  bash

```
export AIRFLOW_HOME=/home/project/airflow
```

Run

4. Submitting a DAG is as simple as copying the DAG python file into `dags` folder in the `AIRFLOW_HOME` directory. Open a terminal and run the command below to submit the DAG.

  plaintext

```
cp dummy_dag.py $AIRFLOW_HOME/dags
```

5. Verify that our DAG actually got submitted. Run the command below to list out all the existing DAGs.

  plaintext

```
airflow dags list
```

6. Verify that `dummy_dag` is a part of the output.

  plaintext

```
airflow dags list | grep dummy_dag
```

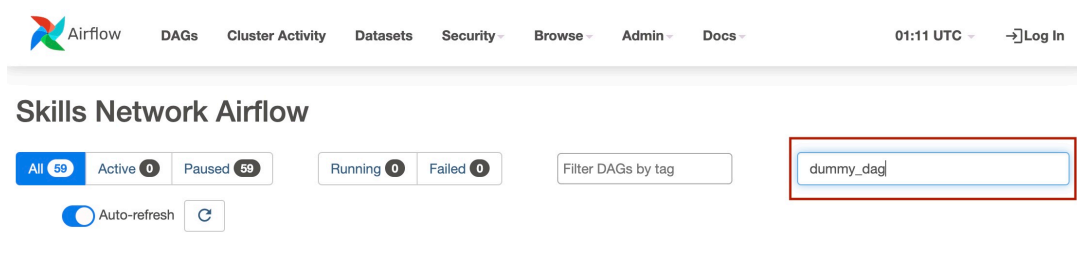
7. Run the command below to list out all the tasks in `dummy_dag`.

  plaintext

You should see 3 tasks in the output.

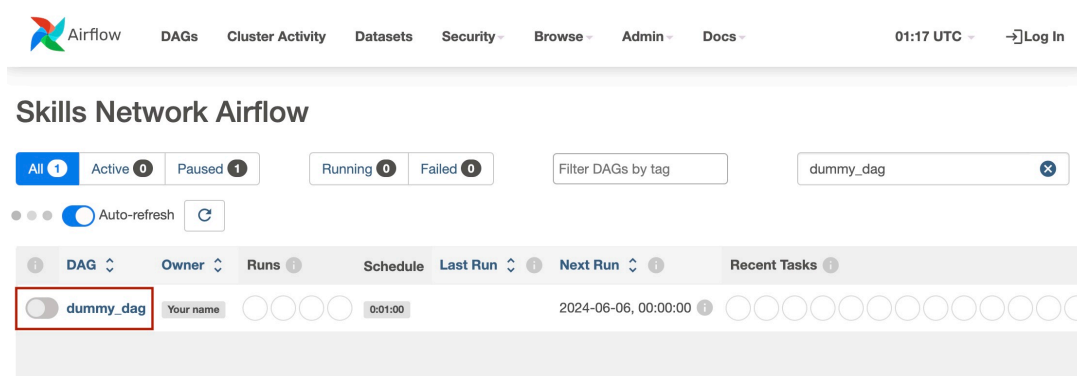
Exercise 4: Search for a DAG

1. In the Web-UI, identify the **Search DAGs** text box as shown in the image below and type **dummy_dag** in the textbox and press enter.



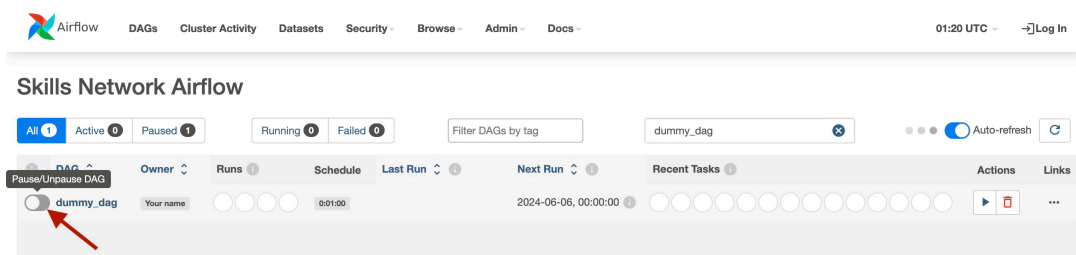
Note: It may take a couple of minutes for the dag to appear here. If you do not see your DAG, please give it a minute and try again.

2. You should see the **dummy_dag** listed as seen in the image below:



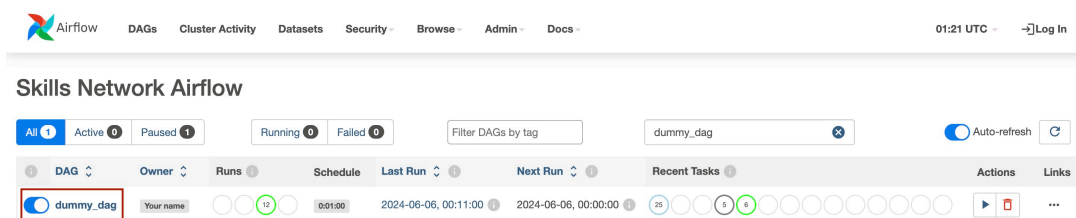
Exercise 5: Pause/Unpause a DAG

1. Unpause the DAG using the Pause/Unpause button.



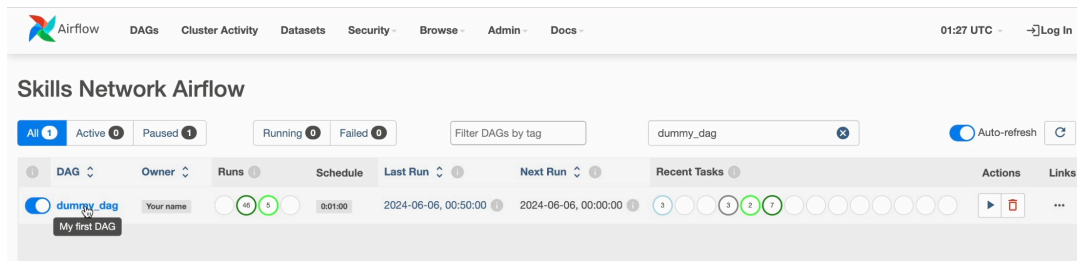
2. You can see the following details in this view.

- Owner of the DAG
- How many times this DAG has run
- Schedule of the DAG
- Last run time of the DAG
- Recent task status

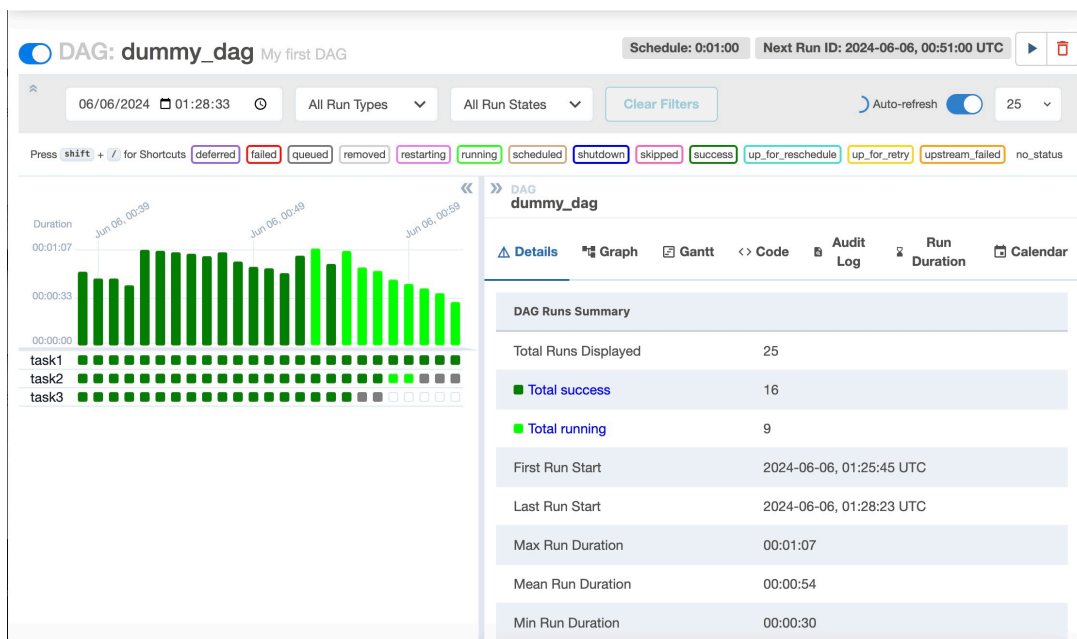


Exercise 6: Detailed view of a DAG

1. Click on the DAG name as shown in the image below to see the detailed view of the DAG.

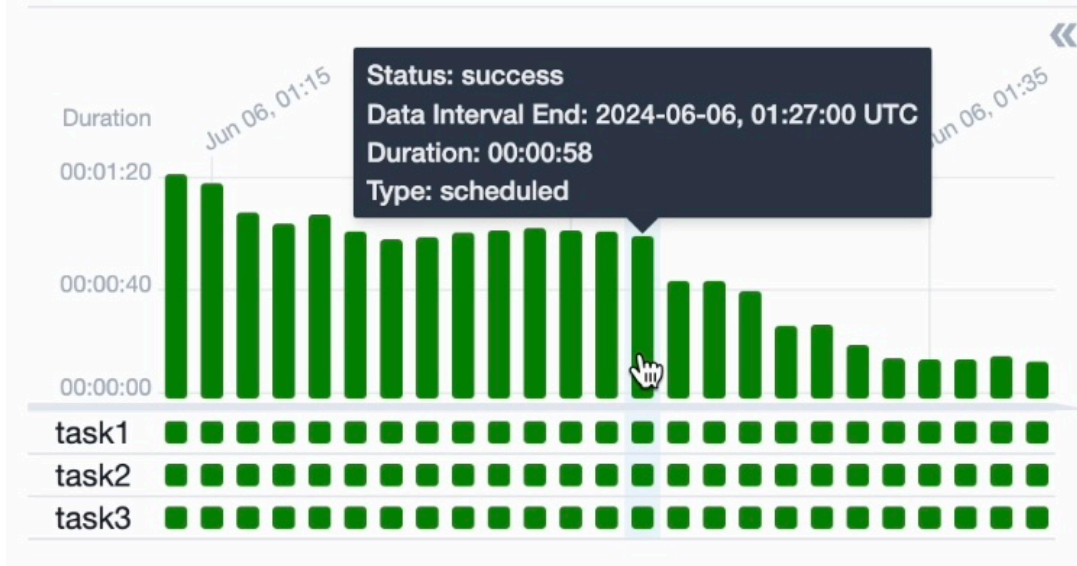


2. You will land on a DAG details page showing the default grid view with the three tasks listed.

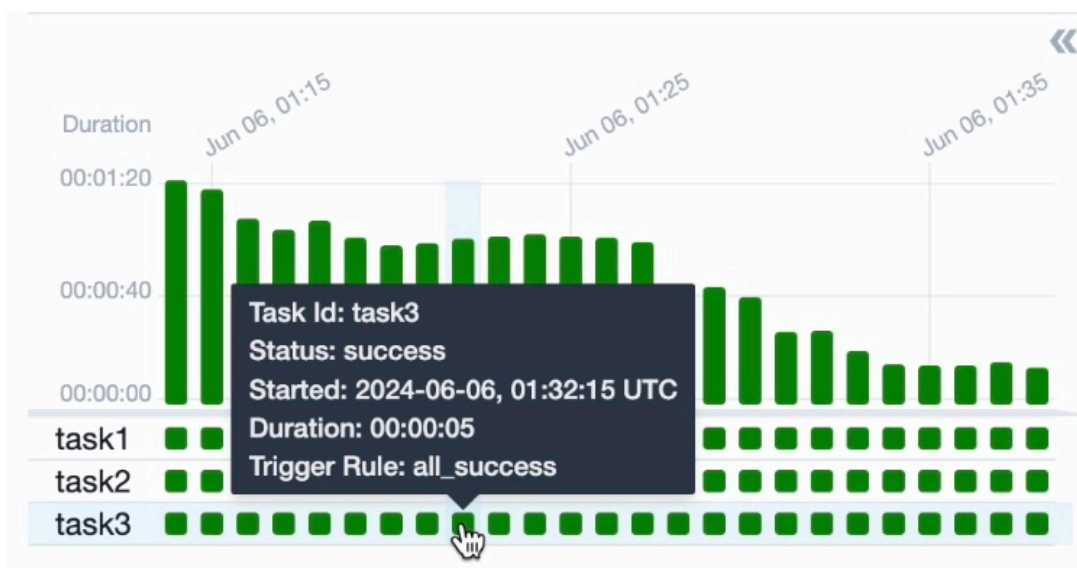


The Grid view shows your DAG tasks in the form of grids as seen in the image. You will observe the **Auto Refresh** button switched on by default on the right corner.

The grids in the image represent a single DAG run and the color indicates the status of the DAG run. Place your mouse on any grid to see the details.

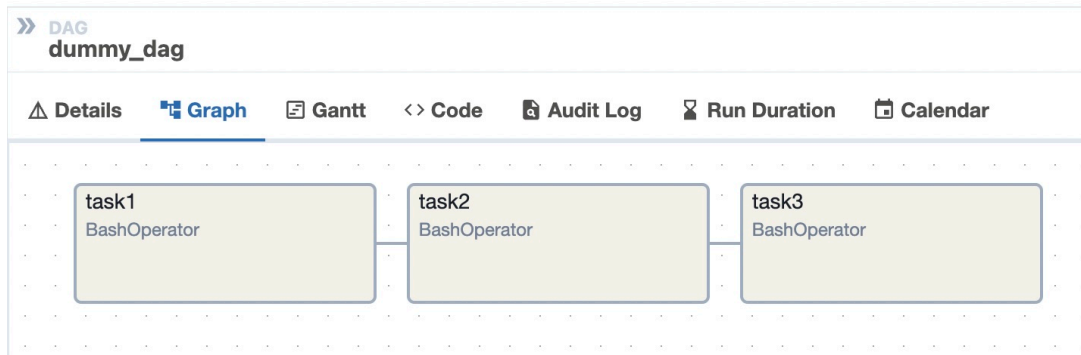


The squares in the image below represent a single task within a DAG run and the color indicates its status. Place your mouse on any square to see the task details.



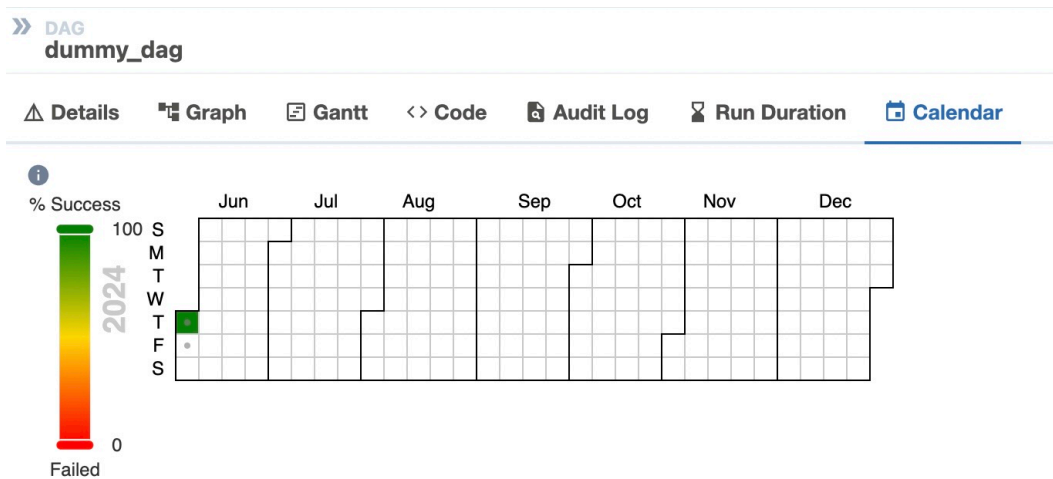
Exercise 7: Explore graph view of DAG

1. Click on the **Graph View** button to open the graph view. The graph view shows the tasks in a form of a graph. With the auto refresh on, each task status is also indicated with the color code.



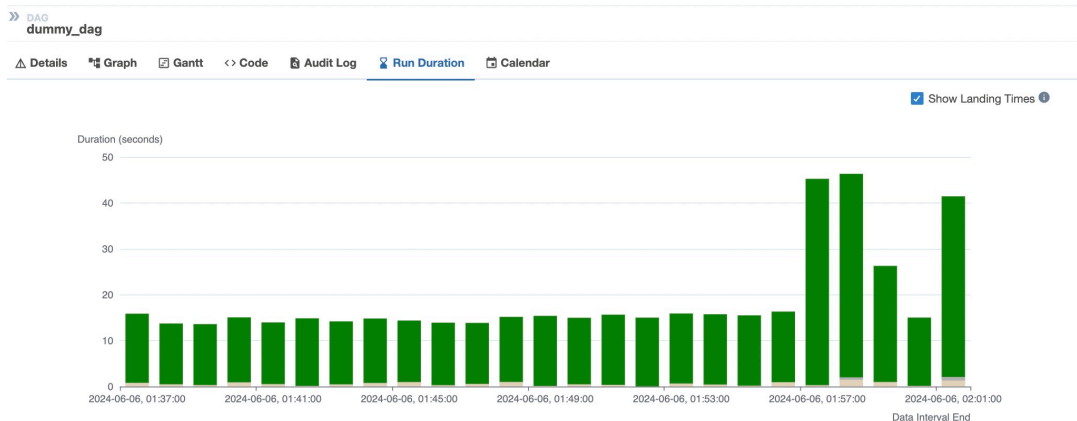
Exercise 8: Calender view

The calender view gives you an overview of all the dates when this DAG was run along with its status as a color code.

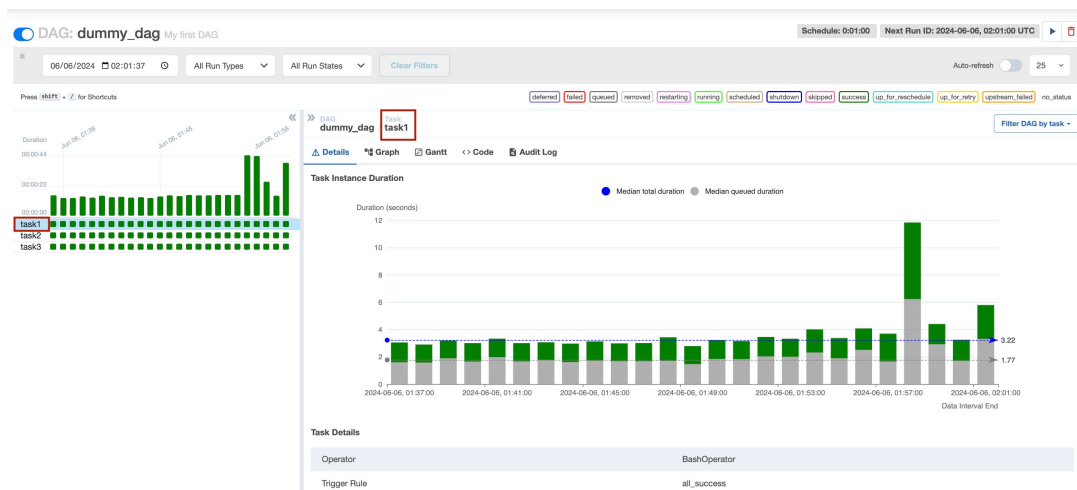


Exercise 9: DAG and Task Duration view

The DAG duration gives you an overview of how much time the entire workflow took.



The Task Duration view gives you an overview of how much time each task took to execute, over a period of time.



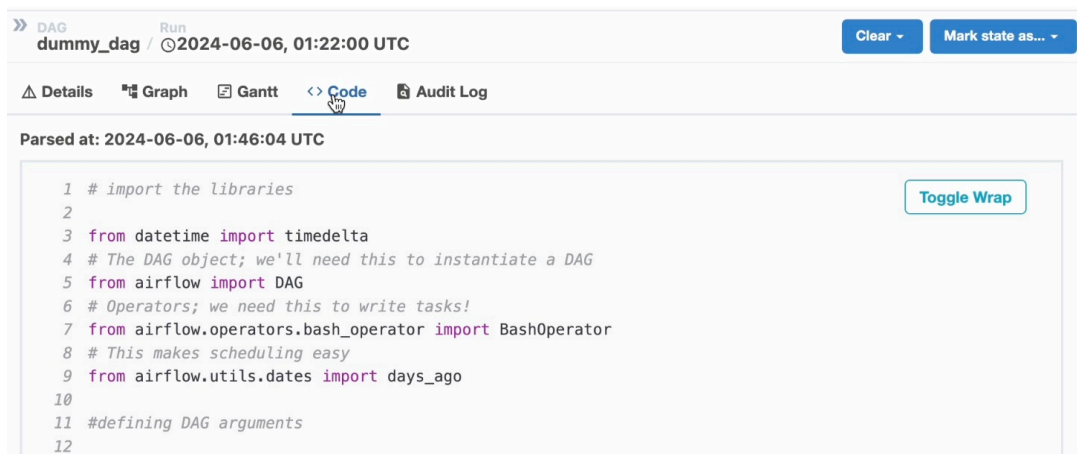
Exercise 10: Details view

The Details view give you all the details of the DAG as specified in the code of the DAG.

» DAG dummy_dag	
Details	Graph Gantt Code Audit Log Run Duration Calendar
DAG Runs Summary	
Total Runs Displayed	25
■ Total success	25
First Run Start	2024-06-06, 01:32:20 UTC
Last Run Start	2024-06-06, 01:54:00 UTC
Max Run Duration	00:00:38
Mean Run Duration	00:00:16
Min Run Duration	00:00:13
DAG Summary	
Total Tasks	3
BashOperators	3
DAG Details	
Dag display name	dummy_dag
Dag id	dummy_dag
Description	My first DAG
Fileloc	/home/project/airflow/dags/dummy_dag.py
Has import errors	false
Has task concurrency limits	false
Is active	true
Is paused	false

Exercise 11: Code view

The Code view lets you view the code of the DAG.



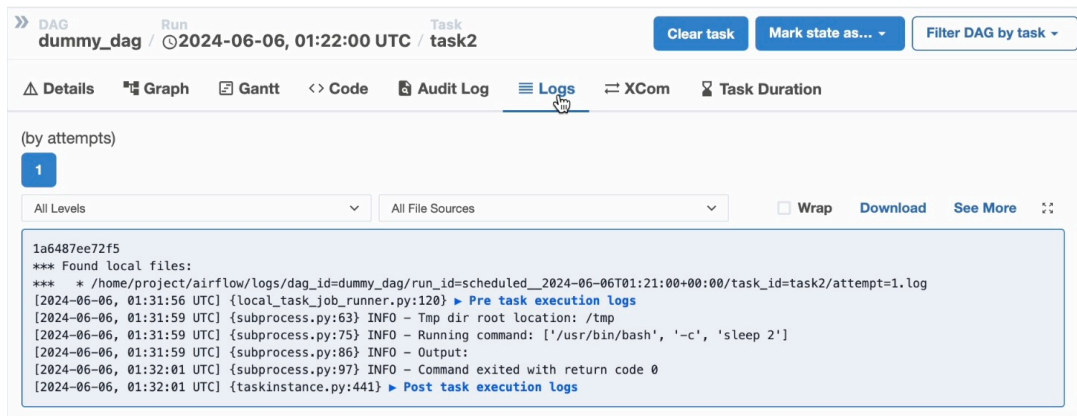
The screenshot shows the Airflow web interface for a DAG named `dummy_dag`. The top bar indicates the DAG is in a `Run` state, with a timestamp of `2024-06-06, 01:22:00 UTC`. There are buttons for `Clear` and `Mark state as...`. Below the top bar, there is a navigation menu with tabs for `Details`, `Graph`, `Gantt`, `Code` (which is selected), and `Audit Log`. The main content area shows the code for the DAG, which is a Python script. The code is as follows:

```
1 # import the libraries
2
3 from datetime import timedelta
4 # The DAG object; we'll need this to instantiate a DAG
5 from airflow import DAG
6 # Operators; we need this to write tasks!
7 from airflow.operators.bash_operator import BashOperator
8 # This makes scheduling easy
9 from airflow.utils.dates import days_ago
10
11 #defining DAG arguments
12
```

On the right side of the code editor, there is a button labeled `Toggle Wrap`.

Exercise 12: Task logs

You can view the logs of an individual task with task logs.

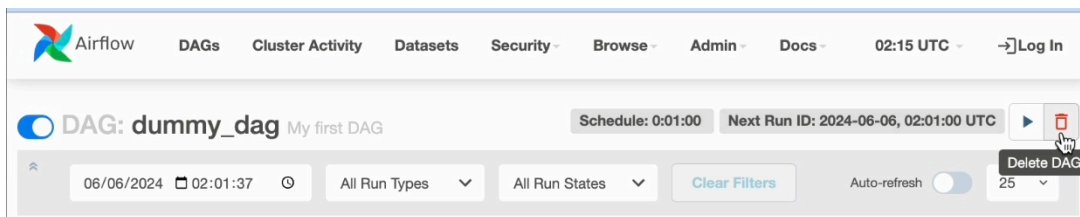


The screenshot displays the Apache Airflow web interface for a task named 'task2' within a 'dummy_dag'. The interface includes a top navigation bar with buttons for 'Clear task', 'Mark state as...', and 'Filter DAG by task'. Below this is a tabbed interface with options like 'Details', 'Graph', 'Gantt', 'Code', 'Audit Log', 'Logs' (which is selected), 'XCom', and 'Task Duration'. The 'Logs' tab shows a list of attempts, with the first attempt (1) selected. Below the attempt list, there are filters for 'All Levels' and 'All File Sources', along with options to 'Wrap', 'Download', and 'See More'. The main log content shows a series of log entries for the task execution, including file discovery, subprocess execution, and command output.

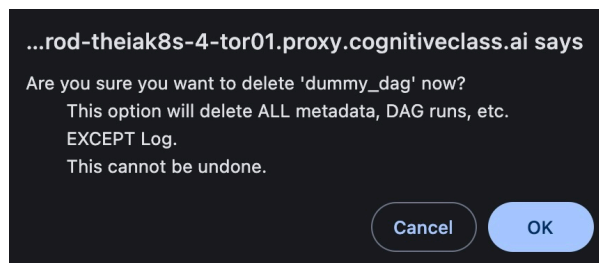
```
1a6487ee72f5
*** Found local files:
*** * /home/project/airflow/logs/dag_id=dummy_dag/run_id=scheduled__2024-06-06T01:21:00+00:00/task_id=task2/attempt=1.log
[2024-06-06, 01:31:56 UTC] {local_task_job_runner.py:120} ▶ Pre task execution logs
[2024-06-06, 01:31:59 UTC] {subprocess.py:63} INFO - Tmp dir root location: /tmp
[2024-06-06, 01:31:59 UTC] {subprocess.py:75} INFO - Running command: ['/usr/bin/bash', '-c', 'sleep 2']
[2024-06-06, 01:31:59 UTC] {subprocess.py:86} INFO - Output:
[2024-06-06, 01:32:01 UTC] {subprocess.py:97} INFO - Command exited with return code 0
[2024-06-06, 01:32:01 UTC] {taskinstance.py:441} ▶ Post task execution logs
```

Exercise 13: Delete a DAG

To delete a DAG click on the delete button.



You will get a confirmation pop up as shown in the image below. Click **OK** to delete the DAG.



Practice exercises

1. Unpause any existing DAG and monitor it.
2. View the details on any existing DAG. View the code of the DAG. Delve into the task details and view the logs of each task.

Authors

[Lavanya T S](#) Ramesh Sannareddy

Other Contributors

Rav Ahuja

© IBM Corporation. All rights reserved.

Practice exercises

1. Unpause any existing DAG and monitor it.
2. View the details on any existing DAG. View the code of the DAG. Delve into the task details and view the logs of each task.

Authors

[Lavanya T S](#) Ramesh Sannareddy

Other Contributors

← Exercise 13: Delete a DAG